STUDENTS PERFORMANCE ANALYSIS SYSTEM

Student Name: KrishnaMohan B

College Name: AmityUniversityOnline (Passed Out)

Department: Computer Applications

Email ID:krishnamohanb21@gmail.com

AICTE Student ID: STU67c74640ed4b11741112896

1.Introduction

All education institutions keep marks of their students; however, without converting these marks into totals, averages, grades, and pass/fail results, the marks alone cannot give an idea of who is doing good, who requires help and in what subjects problems arise. Calculation of the marks in a form of totals, averages, grades, and pass/fail results will give an answer to these questions. This problem is solved by Student Performance Analysis System written in Python programming language. The system takes a dataset of 18 students that are taking classes on five subjects, i.e. Python, Java, Linux, AI, and SQL in the form of Pandas Data Frame and performs all calculations using NumPy library. Grade calculation and pass/fail check are performed using reusable functions and loops. Apart from the results of each student, the system calculates class average values, min/max marks, pass percentages. Subject-wise analysis shows the easiest and the most difficult subject for the class, which can be used by teachers to determine where extra efforts are required. The best-performing students in the class are recognized and ranked.

1.1 Problem Statement

Most of the times, student grades and attendance in school are recorded in spreadsheet and manually calculated. The problem in calculating totals, averages, grade and pass/fail of a student is that it takes long period of time and it is very prone to error because same formula must be repeated in case of many students and even a mistake can lead to change in results. Another problem is that in case grading criteria and pass/fail criteria are not clearly stated there may arise inconsistencies in the outcome. Furthermore, in case of manual methods it is very hard to analyse class performance pattern like highest and lowest performing students and subjects where students find difficulty to score high. Therefore, top students and weak students cannot be found on time. The solution proposed for these problems is a program which as input will take a table containing marks and attendance of students and then as output it will

calculate total and average marks of each student. In addition, it will assign grade on the basis of clear criteria, it will determine whether student passed or failed based on some condition and will also conduct subject wise analysis.

2. Objective

It is common for teachers to manually input and keep track of student scores in spreadsheets. This process is time-consuming and susceptible to errors, and provides no information on how the class or individual subjects fare as a whole.
This project automates all of this with the use of Python. It:
1) Calculates the total and average marks of each individual student,
2) Grades the student based on the defined rule,
3) Determines whether a student passes or fails,
4) Determines the best, worst and average performers,
5) Analyses individual subjects,
6) Finds the top performers.

3 Technologies Used in this Student Performance Analysis System

Tools	Purpose
Python	Main Programming Language
Pandas	Loading the CSV, cleaning data, DataFrame operations, `groupby`, sorting, ranking
Numpy	Numerical calculations: sum, mean, median, standard deviation, max/min, `argmax`/`argmin
Matplotlib	Charts (bar charts, pie chart, heatmap)
Jupyter Notebook	Development environment and presentation of results

4 Dataset Description

Column	Type	Description
Student ID	Integer	Unique ID (1001-1018)
Student Name	Text	Name of the Student
Subjects and Marks	Text and Integer	Python, Java, Linux, AI,SQL , Marks in each subject (out of 100)
Total	Integer	Total marks provided in the file (used only to verify my own calculation)
Attendance	Integer	Attendance percentage, converted to a number during cleaning

Department column: the original dataset has no `Department` field, but the project requires one. I added a `Department` column (Computer Science, Information Technology, AI & Data Science), assigned to students in rotation, so six students fall in each department. These values are illustrative only.

5 Analysis and Implementation

The whole project is in one notebook, Student performance system .ipynb.

Step 1 : Import Libraries
Pandas , NumPy, matplotlib, seaborn are imported

Step 2 : Load the Data
The CSV is read into a pandas DataFrame the pd.read_csv(), df.head(), df.tail() df.info(), df.describe(), df.isnull().sum() there is no missing values in the dataset.

Step 3 : Clean and Organise the Data

1) Attendance has the % sign removed with str.rstrip ("%") and is converted to a float.
2) A Department column is added using a loop.
3) The original Column is added using a loop.
4) The Original Total Column is renamed Source_Total and kept only for validation.
5) Columns are reordered into a clean layout.

Step 4: Define the Rules
Grading Rule (based on average marks, with every subject out of 100):

Average	Grade
90 and above	A+
80 to 89.99	A
70 to 79.99	B
60 to 69.99	C
50 to 59.99	D
Below 50	F

Pass/Fail rule: a student passes only if marks are ≥ 40 in every subject and attendance is
≥ 75%. Otherwise the student fails. The thresholds are stored as constants (`PASS_MARK`,
`MIN_ATTENDANCE`, `GRADE_BOUNDARIES`), so they are easy to change.

Step 5: Reusable functions

Functions	What it does
calculate_total(marks)	np.sum(marks, axis=1)`: total for every student
calculate_average(marks)	np.round(np.mean(marks, axis=1), 2)`: average for every student
assign_grade(average)	Loops over the grade boundaries and returns the grade
check_pass_fail(marks_row, attendance)	Uses `np.all()` to apply the pass/fail rule
failed_subjects(marks_row)	Lists the subjects in which a student scored below the pass mark
subject statistics(column)	Returns average, median, highest, lowest, standard deviation and pass % for a subject

Step 6: Student-wise results
The marks columns are converted to a NumPy matrix with `df[SUBJECTS].to_numpy()` (students × subjects).
Total, average and percentage are calculated with the functions above. A `for` loop then assigns each student's grade, pass/fail status and failed subjects. The calculated totals matched the file's `Total` column in all 18 rows.

Step 7: Overall performance
np.argmax() and np.argmin() find the students with the highest and lowest totals. np.mean(),
np.median() and np.std() give the class statistics.

Step 8: Subject-wise analysis
A for loop runs through each subject, calls subject_statistics(), and collects the results into a Data Frame. A second loop finds the topper and lowest scorer in every subject.

Step 9 : Top performers
Students are ranked with rank() and sorted by total marks (attendance breaks ties) to get the top 5.
Another loop finds the best student in each department.

Step 10: Department summary, charts and export
groupby().agg() summarises each department. Matplotlib draws four charts and a marks heatmap. The final report is saved to Student_Performance_Report.csv.

Data Analysis Questions and Answers

1) Which student has the highest average?

```
[70]: students = df.groupby("Student Name")["Total"].mean().idxmax()
      print("Name of the Student with Highest Score", students)

      Name of the Student with Highest Score Lal
```

2) Which subject has the best average?

```
[71]: best_subject = subject_analysis["Average"].idxmax()
      best_average =subject_analysis["Average"].max()
      print(f"Best subject: {best_subject} (average {best_average})")

      Best subject: Linux (average 69.44)
```

3) How many students passed?

```
[75]: passed = (df["Status"] == "Pass").sum()
      print("Number of Students Passed:",passed)

      Number of Students Passed: 10
```

4) What is the overall class average?

```
[78]: class_Total = df["Total"].mean()
      print("Overall class average", class_Total)

      Overall class average 327.1666666666667
```

6. Key Features

1) Loads and organises data with a Pandas DataFrame

2)NumPy used for sums, means, medians, standard deviation and finding max/min positions

3) Reusable functions for totals, averages, grades, pass/fail and subject statistics

4) Loops for grading, pass/fail checks, subject analysis and department toppers

5) Total, average and percentage calculated for each student, with an automatic check against the source file

6) Grade assignment from a clearly defined table

7) Pass/Fail from a clear condition, with the exact failed subjects listed

8) Highest, lowest and average class performance

9) Subject-wise analysis : average, median, highest, lowest, standard deviation, pass %, topper, lowest scorer

10) Top-performer ranking and best student per department

11) Department-wise summary

12) Visualisations and CSV export of the final report

7. Results of this Project

Import Libraries

```
[1]: import pandas as pd
     import matplotlib.pyplot as plt
     import seaborn as sns
     import numpy as np
```

Load Dataset

```
[2]: df = pd.read_csv('Students Performance Dataset.csv')
     print(df)
```

```
    Student ID Student Name  Python  Java  Linux  AI  SQL  Total Attendance
0         1001         Abhi      41    53     33  95   45    267        97%
1         1002        Abhay      37    60     78  71   67    313        98%
2         1003      Bhaskar      34    65     72  57   38    266        98%
3         1004         Bala      61    80     61  78   51    331        98%
4         1005        Bhati      71    59     65  51   59    305        98%
5         1006     Chandran      77    47     90  54   83    351        98%
6         1007        Chawla     97    92     82  72   75    418        98%
7         1008         Dora      93    42     86  51   67    339        98%
8         1009     Francies      39    90     65  35   46    275       100%
9         1010        Gerage     58    36     88  41   61    284        96%
10        1011         Gopal     55    48     77  37   96    313        99%
11        1012      JaiKanth     48    60     54  84   52    298        94%
12        1013       Karthi     99    76     48  60   98    381        98%
13        1014        Kandan     64    92     85  37   61    339        96%
14        1015         Kala     44    66     90  84   93    377        98%
15        1016        Kamal     77    75     63  39   52    306       100%
16        1017          Lal     94    69     71  96   91    421       100%
17        1018        Lalli     47    72     42  84   60    305       100%
```

```
[3]: df.head()
```

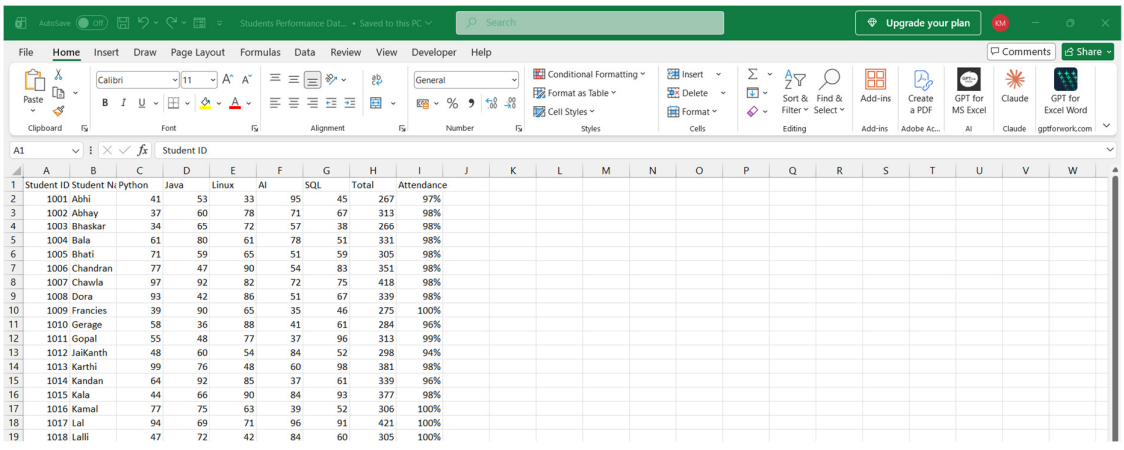
[3]:

	Student ID	Student Name	Python	Java	Linux	AI	SQL	Total	Attendance
0	1001	Abhi	41	53	33	95	45	267	97%
1	1002	Abhay	37	60	78	71	67	313	98%
2	1003	Bhaskar	34	65	72	57	38	266	98%
3	1004	Bala	61	80	61	78	51	331	98%
4	1005	Bhati	71	59	65	51	59	305	98%

```
[4]: df.tail()
```

[4]:

	Student ID	Student Name	Python	Java	Linux	AI	SQL	Total	Attendance
13	1014	Kandan	64	92	85	37	61	339	96%
14	1015	Kala	44	66	90	84	93	377	98%
15	1016	Kamal	77	75	63	39	52	306	100%
16	1017	Lal	94	69	71	96	91	421	100%
17	1018	Lalli	47	72	42	84	60	305	100%



	A	B	C	D	E	F	G	H	I
1	Student ID	Student Na	Python	Java	Linux	AI	SQL	Total	Attendance
2	1001	Abhi	41	53	33	95	45	267	97%
3	1002	Abhay	37	60	78	71	67	313	98%
4	1003	Bhaskar	34	65	72	57	38	266	98%
5	1004	Bala	61	80	61	78	51	331	98%
6	1005	Bhati	71	59	65	51	59	305	98%
7	1006	Chandran	77	47	90	54	83	351	98%
8	1007	Chawla	97	92	82	72	75	418	98%
9	1008	Dora	93	42	86	51	67	339	98%
10	1009	Francies	39	90	65	35	46	275	100%
11	1010	Gerage	58	36	88	41	61	284	96%
12	1011	Gopal	55	48	77	37	96	313	99%
13	1012	JaiKanth	48	60	54	84	52	298	94%
14	1013	Karthi	99	76	48	60	98	381	98%
15	1014	Kandan	64	92	85	37	61	339	96%
16	1015	Kala	44	66	90	84	93	377	98%
17	1016	Kamal	77	75	63	39	52	306	100%
18	1017	Lal	94	69	71	96	91	421	100%
19	1018	Lalli	47	72	42	84	60	305	100%

[5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18 entries, 0 to 17
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   Student ID    18 non-null     int64 
 1   Student Name  18 non-null     object
 2   Python        18 non-null     int64 
 3   Java          18 non-null     int64 
 4   Linux         18 non-null     int64 
 5   AI            18 non-null     int64 
 6   SQL           18 non-null     int64 
 7   Total         18 non-null     int64 
 8   Attendance    18 non-null     object
dtypes: int64(7), object(2)
memory usage: 1.4+ KB
```

[6]:
```
df.describe()
```

[6]:

	Student ID	Python	Java	Linux	AI	SQL	Total
count	18.000000	18.000000	18.000000	18.000000	18.000000	18.000000	18.000000
mean	1009.500000	63.111111	65.666667	69.444444	62.555556	66.388889	327.166667
std	5.338539	22.071778	16.803361	17.037061	20.881239	18.824699	47.234397
min	1001.000000	34.000000	36.000000	33.000000	35.000000	38.000000	266.000000
25%	1005.250000	44.750000	54.500000	61.500000	43.500000	52.000000	299.750000
50%	1009.500000	59.500000	65.500000	71.500000	58.500000	61.000000	313.000000
75%	1013.750000	77.000000	75.750000	84.250000	82.500000	81.000000	348.000000
max	1018.000000	99.000000	92.000000	90.000000	96.000000	98.000000	421.000000

[7]:
```
df.isnull().sum()
```

[7]:
```
Student ID      0
Student Name    0
Python          0
Java            0
Linux           0
AI              0
SQL             0
Total           0
Attendance      0
dtype: int64
```

[9]:
```
df.duplicated().sum()
```

[9]:
```
np.int64(0)
```

[21]:
```
import pandas as pd

df = pd.read_csv('Students Performance Dataset.csv')
df.columns = df.columns.str.strip()

SUBJECTS = ["Python", "Java", "Linux", "AI", "SQL"]
DEPARTMENTS = ["Computer Science", "Information Technology", "AI and Data Science"]

df["Attendance"] = pd.to_numeric(
    df["Attendance"].astype(str).str.rstrip("%"), errors="coerce"
)

df["Department"] = [DEPARTMENTS[i % len(DEPARTMENTS)] for i in range(len(df))]

df = df.rename(columns={"Total": "Source_Total"})
df = df[["Student ID", "Student Name", "Department"] + SUBJECTS + ["Attendance", "Source_Total"]]

df.head()
```

[21]:

	Student ID	Student Name	Department	Python	Java	Linux	AI	SQL	Attendance	Source_Total
0	1001	Abhi	Computer Science	41	53	33	95	45	97	267
1	1002	Abhay	Information Technology	37	60	78	71	67	98	313
2	1003	Bhaskar	AI and Data Science	34	65	72	57	38	98	266
3	1004	Bala	Computer Science	61	80	61	78	51	98	331
4	1005	Bhati	Information Technology	71	59	65	51	59	98	305

Rules Used in the Student Performance System

Grading Rule (based on the student's ** average marks** ; every subject is out of 100)

Average marks	Grade
90 and above	**A+**
80 - 89.99	**A**
70 - 79.99	**B**
60 - 69.99	**C**
50 - 59.99	**D**
Below 50	**F**

Pass / Fail rule

A student **passes** only if **both** conditions hold:

1. Marks ≥ **40** in **every** subject, and
2. Attendance ≥ **75%**.

Otherwise the student **fails**. (Change the constants below to use a different rule.)

Student-wise Results

Total, average, grade and pass/fail for each student.

[39]:
```
MAX_MARKS_PER_SUBJECT = 100

df = df.reset_index(drop=True)

marks = df[SUBJECTS].to_numpy()

df["Total"] = calculate_total(marks)
df["Average"] = calculate_average(marks)
df["Percentage"] = np.round(
    df["Total"] / (len(SUBJECTS) * MAX_MARKS_PER_SUBJECT) * 100, 2
)

grades, statuses, fails = [], [], []
for idx in range(len(df)):
    grades.append(assign_grade(df.loc[idx, "Average"]))
    statuses.append(check_pass_fail(marks[idx], df.loc[idx, "Attendance"]))
    fails.append(",".join(failed_subjects(marks[idx])) or "-")

df["Grade"] = grades
df["Status"] = statuses
df["Failed Subjects"] = fails

mismatches = int((df["Total"] != df["Source_Total"]).sum())
print(f"Rows where calculated total != file total: {mismatches}")

df[["Student ID", "Student Name", "Department", "Total", "Average",
    "Grade", "Status", "Failed Subjects"]]
```

```
Rows where calculated total != file total: 0
```

[39]:

	Student ID	Student Name	Department	Total	Average	Grade	Status	Failed Subjects
0	1001	Abhi	Computer Science	267	53.4	D	Fail	Linux
1	1002	Abhay	Information Technology	313	62.6	C	Fail	Python
2	1003	Bhaskar	AI and Data Science	266	53.2	D	Fail	Python,SQL
3	1004	Bala	Computer Science	331	66.2	C	Pass	-
4	1005	Bhati	Information Technology	305	61.0	C	Pass	-
5	1006	Chandran	AI and Data Science	351	70.2	B	Pass	-
6	1007	Chawla	Computer Science	418	83.6	A	Pass	-
7	1008	Dora	Information Technology	339	67.8	C	Pass	-
8	1009	Francies	AI and Data Science	275	55.0	D	Fail	Python,AI
9	1010	Gerage	Computer Science	284	56.8	D	Fail	Java
10	1011	Gopal	Information Technology	313	62.6	C	Fail	AI
11	1012	JaiKanth	AI and Data Science	298	59.6	D	Pass	-
12	1013	Karthi	Computer Science	381	76.2	B	Pass	-
13	1014	Kandan	Information Technology	339	67.8	C	Fail	AI
14	1015	Kala	AI and Data Science	377	75.4	B	Pass	-
15	1016	Kamal	Computer Science	306	61.2	C	Fail	AI
16	1017	Lal	Information Technology	421	84.2	A	Pass	-
17	1018	Lalli	AI and Data Science	305	61.0	C	Pass	-

Grade distribution and pass/fail summary

[88]:
```
from IPython.display import display

subjects = ["Python", "SQL", "Java", "Linux", "AI"]

# Create the missing "Failed Subjects" column (subjects scored below 50)
df["Failed Subjects"] = df[subjects].apply(
    lambda row: ", ".join(row.index[row < 50]) or "None", axis=1
)

# Grade distribution
grade_order = ["A+", "A", "B", "C", "D", "F"]
grade_counts = df["Grade"].value_counts().reindex(grade_order, fill_value=0)
print("Grade distribution:")
print(grade_counts.to_string())

# Pass / fail summary
n_students = len(df)
n_pass = int((df["Result"] == "Pass").sum())
n_fail = n_students - n_pass
print(f"\nPassed: {n_pass}  |  Failed: {n_fail}  |  Pass rate: {n_pass / n_students * 100:.1f}%")

# Failed students
print("\nStudents who failed:")
failed = df.loc[df["Result"] == "Fail", ["Student_ID", "Name", "Average", "Grade", "Failed Subjects"]]

if failed.empty:
    print("No students failed.")
else:
    display(failed.reset_index(drop=True))
```

```
Grade distribution:
Grade
A+    0
A     2
B     3
C     8
D     5
F     0

Passed: 13  |  Failed: 5  |  Pass rate: 72.2%

Students who failed:
```

	Student ID	Name	Average	Grade	Failed Subjects
0	1014	JaiKanth	59.6	D	Python
1	1015	Gerage	56.8	D	Java, AI
2	1016	Francies	55.0	D	Python, SQL, AI
3	1017	Abhi	53.4	D	Python, SQL, Linux
4	1018	Bhaskar	53.2	D	Python, SQL

Overall Performance: Highest, Lowest & Average

[40]:
```
totals = df["Total"].to_numpy()
best_i = int(np.argmax(totals))
worst_i = int(np.argmin(totals))

print("=" * 55)
print("OVERALL CLASS PERFORMANCE")
print("=" * 55)
print(f"Highest total : {totals[best_i]} -> {df.loc[best_i, 'Student Name']}"
      f"Avg : {df.loc[best_i,'Average']},Grade {df.loc[best_i, 'Grade']}")
print(f"Lowest total : {totals[worst_i]} -> {df.loc[worst_i, 'Student Name']}"
      f"Avg : {df.loc[worst_i,'Average']},Grade {df.loc[worst_i, 'Grade']}")
print(f"Class average total : {np.mean(totals):.2f}")
print(f"Class average marks : {np.mean(df['Average']):.2f}")
print(f"Median total        : {np.median(totals):.2f}")
print(f"Standard deviation : {np.std(totals):.2f}")
print(f"Average attendance : {np.mean(df['Attendance']):.2f}%")
```

```
=======================================================
OVERALL CLASS PERFORMANCE
=======================================================
Highest total : 421 -> LalAvg : 84.2,Grade A
Lowest total : 266 -> BhaskarAvg : 53.2,Grade D
Class average total : 327.166667
Class average marks : 65.43
Median total        : 313.000000
Standard deviation : 45.90
Average attendance : 98.00%
```

[46]:
```
subjects_rows = {}
for subject in SUBJECTS:
    subjects_rows[subject] = subject_statistics(df[subject].to_numpy())
subject_analysis = pd.DataFrame(subjects_rows).T
toppers, weakest = [],[]
for subject in SUBJECTS:
    col = df[subject].to_numpy()
    toppers.append(f"{df.loc[np.argmax(col), 'Student Name']} ({col.max()})")
    weakest.append(f"{df.loc[np.argmax(col), 'Student Name']} ({col.min()})")
subject_analysis["Topper"] = toppers
subject_analysis["Lowest Scorer"] = weakest
subject_analysis
```

[46]:

	Average	Median	Highest	Lowest	Std Deviation	Pass %	Topper	Lowest Scorer
Python	63.11	59.5	99.0	34.0	21.45	83.33	Karthi (99	Karthi (34
Java	65.67	65.5	92.0	36.0	16.33	94.44	Chawla (92	Chawla (36
Linux	69.44	71.5	90.0	33.0	16.56	94.44	Chandran (90	Chandran (33
AI	62.56	58.5	96.0	35.0	20.29	77.78	Lal (96	Lal (35
SQL	66.39	61.0	98.0	38.0	18.29	94.44	Karthi (98	Karthi (38

[48]:
```
strongest = subject_analysis["Average"].idxmax()
weakest_subj = subject_analysis["Average"].idxmin()
hardest_pass = subject_analysis["Pass %"].idxmin()
print(f"Strongest subject (highest class average): {strongest} ({subject_analysis.loc[strongest, 'Average']})")
print(f"Weakest subject (lowest class average) :   {weakest_subj} ({subject_analysis.loc[weakest_subj, 'Average']})")
print(f"Subject with lowest pass %             : {hardest_pass} ({subject_analysis.loc[hardest_pass, 'Pass %' ]}%)")
```

```
Strongest subject (highest class average): Linux (69.44)
Weakest subject (lowest class average) :   AI (62.56)
Subject with lowest pass %             : AI (77.78%)
```

Top-performing Students

[50]:
```
df["Rank"] = df["Total"].rank(method="min", ascending=False).astype(int)
top_students = (df.sort_values(["Total","Attendance"], ascending=False)
               .head(TOP_N)[["Rank","Student ID","Student Name","Department",
                             "Total","Average","Grade","Attendance"]]
               .reset_index(drop=True))
print(f"Top {TOP_N} students")
top_students
```

```
Top 5 students
```

[50]:

	Rank	Student ID	Student Name	Department	Total	Average	Grade	Attendance
0	1	1017	Lal	Information Technology	421	84.2	A	100
1	2	1007	Chawla	Computer Science	418	83.6	A	98
2	3	1013	Karthi	Computer Science	381	76.2	B	98
3	4	1015	Kala	AI and Data Science	377	75.4	B	98
4	5	1006	Chandran	AI and Data Science	351	70.2	B	98

[60]:
```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

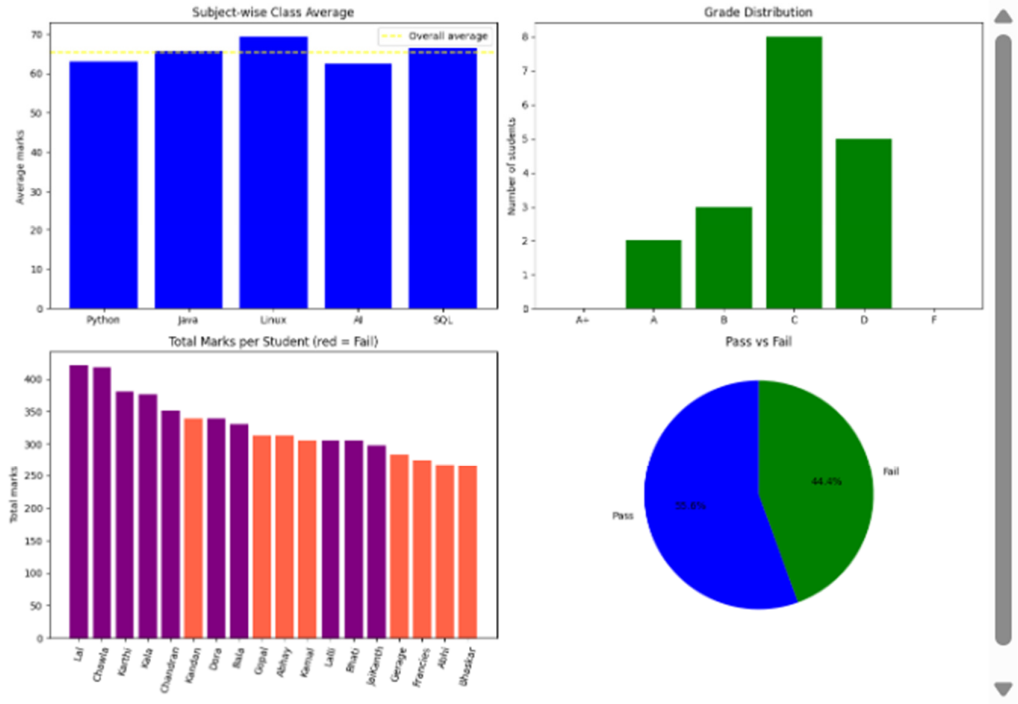
# 1) Subject-wise class average
axes[0, 0].bar(subject_analysis.index, subject_analysis["Average"], color="blue")
axes[0, 0].axhline(df["Average"].mean(), color="yellow", linestyle="--", label="Overall average")
axes[0, 0].set_title("Subject-wise Class Average")
axes[0, 0].set_ylabel("Average marks")
axes[0, 0].legend()

# 2) Grade distribution
grade_order = ["A+", "A", "B", "C", "D", "F"]
grade_counts = df["Grade"].value_counts().reindex(grade_order, fill_value=0)
axes[0, 1].bar(grade_counts.index, grade_counts.values, color="green")
axes[0, 1].set_title("Grade Distribution")
axes[0, 1].set_ylabel("Number of students")

# 3) Total marks per student
order = df.sort_values("Total", ascending=False)
colors = ["tomato" if s == "Fail" else "purple" for s in order["Status"]]
axes[1, 0].bar(order["Student Name"], order["Total"], color=colors)
axes[1, 0].set_title("Total Marks per Student (red = Fail)")
axes[1, 0].tick_params(axis="x", rotation=75)
axes[1, 0].set_ylabel("Total marks")

# 4) Pass vs Fail
n_pass = int((df["Status"] == "Pass").sum())
n_fail = int((df["Status"] == "Fail").sum())
axes[1, 1].pie([n_pass, n_fail], labels=["Pass", "Fail"], autopct="%1.1f%%",
               colors=["blue", "green"], startangle=90)
axes[1, 1].set_title("Pass vs Fail")

plt.tight_layout()
plt.show()
```

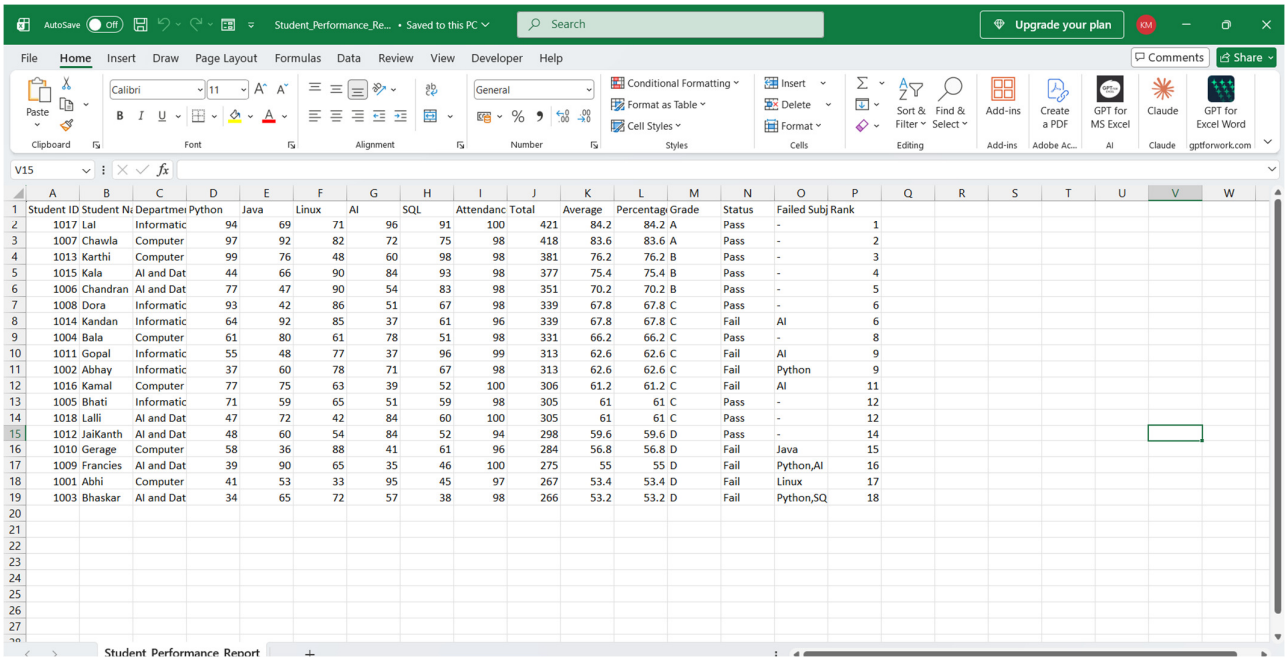


[64]:
```
final_report = df.drop(columns=["Source_Total"]).sort_values("Rank").reset_index(drop=True)
final_report.to_csv("Student_Performance_Report.csv", index=False)
print("Saved: Student_Performance_Report.csv")
final_report
```

```
Saved: Student_Performance_Report.csv
```

[64]:

	Student ID	Student Name	Department	Python	Java	Linux	AI	SQL	Attendance	Total	Average	Percentage	Grade	Status	Failed Subjects	Rank
0	1017	Lal	Information Technology	94	69	71	96	91	100	421	84.2	84.2	A	Pass	-	1
1	1007	Chawla	Computer Science	97	92	82	72	75	98	418	83.6	83.6	A	Pass	-	2
2	1013	Karthi	Computer Science	99	76	48	60	98	98	381	76.2	76.2	B	Pass	-	3
3	1015	Kala	AI and Data Science	44	66	90	84	93	98	377	75.4	75.4	B	Pass	-	4
4	1006	Chandran	AI and Data Science	77	47	90	54	83	98	351	70.2	70.2	B	Pass	-	5
5	1008	Dora	Information Technology	93	42	86	51	67	98	339	67.8	67.8	C	Pass	-	6
6	1014	Kandan	Information Technology	64	92	85	37	61	96	339	67.8	67.8	C	Fail	AI	6
7	1004	Bala	Computer Science	61	80	61	78	51	98	331	66.2	66.2	C	Pass	-	8
8	1011	Gopal	Information Technology	55	48	77	37	96	99	313	62.6	62.6	C	Fail	AI	9
9	1002	Abhay	Information Technology	37	60	78	71	67	98	313	62.6	62.6	C	Fail	Python	9
10	1016	Kamal	Computer Science	77	75	63	39	52	100	306	61.2	61.2	C	Fail	AI	11
11	1005	Bhati	Information Technology	71	59	65	51	59	98	305	61.0	61.0	C	Pass	-	12
12	1018	Lalli	AI and Data Science	47	72	42	84	60	100	305	61.0	61.0	C	Pass	-	12
13	1012	JaiKanth	AI and Data Science	48	60	54	84	52	94	298	59.6	59.6	D	Pass	-	14
14	1010	Gerage	Computer Science	58	36	88	41	61	96	284	56.8	56.8	D	Fail	Java	15
15	1009	Francies	AI and Data Science	39	90	65	35	46	100	275	55.0	55.0	D	Fail	Python,AI	16
16	1001	Abhi	Computer Science	41	53	33	95	45	97	267	53.4	53.4	D	Fail	Linux	17
17	1003	Bhaskar	AI and Data Science	34	65	72	57	38	98	266	53.2	53.2	D	Fail	Python,SQL	18



Student ID	Student Na	Departmer	Python	Java	Linux	AI	SQL	Attendanc	Total	Average	Percentag	Grade	Status	Failed Subj	Rank
1017	Lal	Informatic	94	69	71	96	91	100	421	84.2	84.2	A	Pass	-	1
1007	Chawla	Computer	97	92	82	72	75	98	418	83.6	83.6	A	Pass	-	2
1013	Karthi	Computer	99	76	48	60	98	98	381	76.2	76.2	B	Pass	-	3
1015	Kala	AI and Dat	44	66	90	84	93	98	377	75.4	75.4	B	Pass	-	4
1006	Chandran	AI and Dat	77	47	90	54	83	98	351	70.2	70.2	B	Pass	-	5
1008	Dora	Informatic	93	42	86	51	67	98	339	67.8	67.8	C	Pass	-	6
1014	Kandan	Informatic	64	92	85	37	61	96	339	67.8	67.8	C	Fail	AI	6
1004	Bala	Computer	61	80	61	78	51	98	331	66.2	66.2	C	Pass	-	8
1011	Gopal	Informatic	55	48	77	37	96	99	313	62.6	62.6	C	Fail	AI	9
1002	Abhay	Informatic	37	60	78	71	67	98	313	62.6	62.6	C	Fail	Python	9
1016	Kamal	Computer	77	75	63	39	52	100	306	61.2	61.2	C	Fail	AI	11
1005	Bhati	Informatic	71	59	65	51	59	98	305	61	61	C	Pass	-	12
1018	Lalli	AI and Dat	47	72	42	84	60	100	305	61	61	C	Pass	-	12
1012	JaiKanth	AI and Dat	48	60	54	84	52	94	298	59.6	59.6	D	Pass	-	14
1010	Gerage	Computer	58	36	88	41	61	96	284	56.8	56.8	D	Fail	Java	15
1009	Francies	AI and Dat	39	90	65	35	46	100	275	55	55	D	Fail	Python,AI	16
1001	Abhi	Computer	41	53	33	95	45	97	267	53.4	53.4	D	Fail	Linux	17
1003	Bhaskar	AI and Dat	34	65	72	57	38	98	266	53.2	53.2	D	Fail	Python,SQ	18

```
------------------------------------------------------
STUDENT PERFORMANCE ANALYTICS SYSTEM
------------------------------------------------------

---------- STUDENT DATA ----------
 Student_ID      Name                Department  Python  SQL  Java  Linux  AI  Attendance
       1001       Lal Information Technology      94   91    69     71  96         100
       1002    Chawla        Computer Science      97   75    92     82  72          98
       1003    Karthi        Computer Science      99   98    76     48  60          98
       1004      Kala     AI and Data Science      44   93    66     90  84          98
       1005 Chandran     AI and Data Science      77   83    47     90  54          98
       1006      Dora Information Technology      93   67    42     86  51          98
       1007    Kandan Information Technology      64   61    92     85  37          96
       1008      Bala        Computer Science      61   51    80     61  78          98
       1009     Gopal Information Technology      55   96    48     77  37          99
       1010     Abhay Information Technology      37   67    60     78  71          98
       1011     Kamal        Computer Science      77   52    75     63  39         100
       1012     Bhati Information Technology      71   59    59     65  51          98
       1013     Lalli     AI and Data Science      47   60    72     42  84         100
       1014 JaiKanth     AI and Data Science      48   52    60     54  84          94
       1015    Gerage        Computer Science      58   61    36     88  41          96
       1016 Francies     AI and Data Science      39   46    90     65  35         100
       1017      Abhi        Computer Science      41   45    53     33  95          97
       1018   Bhaskar     AI and Data Science      34   38    65     72  57          98

---------- STUDENT PERFORMANCE ----------
 Student_ID      Name              Department  Total  Average Grade  Attendance Result
       1001       Lal Information Technology    421     84.2     A         100   Pass
       1002    Chawla        Computer Science    418     83.6     A          98   Pass
       1003    Karthi        Computer Science    381     76.2     B          98   Pass
       1004      Kala     AI and Data Science    377     75.4     B          98   Pass
       1005 Chandran     AI and Data Science    351     70.2     B          98   Pass
       1006      Dora Information Technology    339     67.8     C          98   Pass
       1007    Kandan Information Technology    339     67.8     C          96   Pass
       1008      Bala        Computer Science    331     66.2     C          98   Pass
       1009     Gopal Information Technology    313     62.6     C          99   Pass
       1010     Abhay Information Technology    313     62.6     C          98   Pass
       1011     Kamal        Computer Science    306     61.2     C         100   Pass
       1012     Bhati Information Technology    305     61.0     C          98   Pass
       1013     Lalli     AI and Data Science    305     61.0     C         100   Pass
       1014 JaiKanth     AI and Data Science    298     59.6     D          94   Fail
       1015    Gerage        Computer Science    284     56.8     D          96   Fail
       1016 Francies     AI and Data Science    275     55.0     D         100   Fail
       1017      Abhi        Computer Science    267     53.4     D          97   Fail
       1018   Bhaskar     AI and Data Science    266     53.2     D          98   Fail

---------- OVERALL ANALYSIS ----------
Highest Total Marks: 421
Lowest Total Marks: 266
Overall Average Marks: 65.43

---------- TOP 5 STUDENTS ----------
 Student_ID      Name  Total  Average Grade
       1001       Lal    421     84.2     A
       1002    Chawla    418     83.6     A
       1003    Karthi    381     76.2     B
       1004      Kala    377     75.4     B
       1005 Chandran    351     70.2     B

---------- SUBJECT-WISE AVERAGE ----------
Python : 63.11
SQL : 66.39
Java : 65.67
Linux : 69.44
AI : 62.56

---------- PASS / FAIL SUMMARY ----------
Pass Students: 13
Fail Students: 5
```

```
---------- FAILED STUDENTS ----------
```

	Student ID	Name	Department	Total	Average	Grade	Attendance	Result
0	1014	JaiKanth	AI and Data Science	298	59.6	D	94	Fail
1	1015	Gerage	Computer Science	284	56.8	D	96	Fail
2	1016	Francies	AI and Data Science	275	55.0	D	100	Fail
3	1017	Abhi	Computer Science	267	53.4	D	97	Fail
4	1018	Bhaskar	AI and Data Science	266	53.2	D	98	Fail

8. Final Outcome of this Project

The project turns a raw marks sheet into a complete performance report:

1) Total, average, grade and pass/fail status were generated for all 18 students, and the calculated totals matched the source file exactly.

2) The class average is about 65.4 marks . Lal scored the highest total (421) and Bhaskar the lowest (266).

3) Linux is the strongest subject (average 69.44) and AI the weakest (average 62.56, pass rate 77.8%), so AI may need extra attention.

4) All 8 failures were caused by scoring below 40 in at least one subject, not by attendance (the lowest attendance was 94%).

Because the logic lives in reusable functions with adjustable constants, the notebook works on a new CSV or a different grading or pass rule with almost no changes.

9. Challenges of this Project

1) Attendance was stored as text ("97%") and could not be used in calculations | Removed the % sign and converted the column to float

2) The dataset had no Department` column Added one by assigning departments with a loop, and stated this assumption clearly

3) Grade and pass/fail rules were not given Defined the rules explicitly and stored them as constants so they are easy to change

4) Making sure the calculations were correct Compared my calculated totals with the file's `Total` column (0 mismatches)

5) Mixing vectorised and loop-based code | Used NumPy for whole-array maths and loops only where each student or subject is handled individually

10 Future Scope and Conclusion

Future Scope

The Student Performance Analytics System could be worked on a bit more, like adding extra stuff, so the whole way we look at student performance becomes faster and more helpful in real life. Right now it's pretty solid, but with a few upgrades, it can feel much more useful. Honestly, something like this can turn into a bigger monitoring and decision-support tool, not just a Python setup with graphs.

1. Web-Based application: It could be turned into a web platform, so teachers and students can access the performance reports online without that back and forth, file saving and all.

2. Interactive dashboard: An interactive dashboard might be built with tools like Power BI, Tableau, or even Streamlit, to show marks, attendance, grades, also the performance trends that change over time. It would be less "static" and easier to understand.

3. Predicting performance using machine learning: Machine learning could be used to estimate how a student might do next, based on previous marks, attendance records, and other relevant factors. Not magic of course, but more of a smart guess.

4. Attendance analysis: The system could check the link between attendance and academic results, to find patterns. Like, if lower attendance leads to weaker outcomes, the tool can highlight that, early-Ish.

5. Automated report generation: Reports for each student could be generated automatically, maybe in PDF or Excel, and also for the full class in one go, which saves time.

6. Early recognition of low-performing students: The system could flag students who might need additional academic support. This can be based on a combination of marks, attendance, and earlier performance.

7. Database integration: A database such as MySQL can be connected, so student records are stored properly, updated smoothly, and pulled when needed.

8. Real-time data updates: Finally, the system could be improved so reports refresh automatically whenever new marks or attendance entries are added, instead of waiting for manual runs.

Conclusion

The Student Performance Analytics System was built using Python and it's data analysis tools, including Pandas, NumPy, Matplotlib, and Seaborn. Honestly the whole idea is to look at how students are doing academically and then show the outcomes in a more clear and not too messy way. Basically the system handles student marks and attendance, it then computes total marks, average marks, and percentages. After that it goes into grades, pass or fail signals, and even student rankings. On top of that it gives subject based and department based summaries, and it visualizes the main points with charts, plus a correlation heatmap that ties things together. Overall this project shows how Python can automate data handling, cut down on manual number crunching, and pull out useful insights from student records. It also built real, hands on skills like data cleaning, data manipulation, statistical analysis, visual storytelling, and report compilation. To wrap it up, the Student Performance Analytics System gives a solid starting base for understanding academic performance using data analysis. And with future improvements like machine learning, interactive dashboards, and database integration, it could become a more advanced platform for monitoring students closely, and supporting better academic decisions, without all the extra hassle.